MON APPRENTISSAGE — SERIES PYTHON

# Python

## Du zéro à l'expert

## Chapitre 26 — APIs et Web avec Python

*requests, Flask, FastAPI — consommer et créer des services web*

**Auteur :**

**KETOTSA AMÉVI CLAUDE**

**Date de publication :**

24 mars 2026

*Publié dans le cadre de mon parcours d'apprentissage Python*

# Table des matières

**Chapitre 26 — APIs et Web avec Python** **2**
0.1 Le protocole HTTP — Les bases . . . . . 2
0.2 requests — Consommer une API . . . . . 2
0.3 Flask — Créer une API REST . . . . . 4
0.4 FastAPI — L'API moderne . . . . . 7
0.5 Web scraping — Extraire des données du web . . . . . 9
0.6 Ce qui m'a le plus surpris dans ce chapitre . . . . . 10
0.7 Résumé du Chapitre 26 . . . . . 11

# Chapitre 26 — APIs et Web avec Python

**À retenir**

Une **API** (Application Programming Interface) est une interface qui permet à des programmes de communiquer entre eux via le réseau. Les APIs REST utilisent HTTP — le protocole du web — pour échanger des données JSON. Python excelle dans ce domaine avec **requests** pour consommer des APIs, **Flask** pour en créer rapidement, et **FastAPI** pour des services modernes, performants et auto-documentés. Ce chapitre couvre les deux faces du web : client et serveur.

## Le protocole HTTP — Les bases

| GET | POST | PUT | PATCH | DELETE |
|---|---|---|---|---|
| Lire des données | Créer une ressource | Remplacer une ressource | Modifier partiellement | Supprimer |

## requests — Consommer une API

**requests — appels HTTP**

```python
import requests
import json

# GET      lire des donnees
response = requests.get(
    "https://jsonplaceholder.typicode.com/users"
)

print(f"Statut   : {response.status_code}")  # 200
print(f"Type     : {response.headers['Content-Type']}")
print(f"Longueur : {len(response.content)} octets")

# Convertir en Python
utilisateurs = response.json()   # liste de dicts
print(f"Nombre d'utilisateurs : {len(utilisateurs)}")
```

```python
for u in utilisateurs[:3]:
    print(f"  {u['name']:25} | {u['email']}")

# GET avec parametres
params = {"userId": 1, "_limit": 5}
posts = requests.get(
    "https://jsonplaceholder.typicode.com/posts",
    params=params
).json()
print(f"\nPosts de l'user 1 : {len(posts)}")

# POST     envoyer des donnees
nouveau_post = {
    "title":  "Mon apprentissage Python",
    "body":   "Chapitre 26     APIs et Web",
    "userId": 1
}
resp = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=nouveau_post   # encode en JSON automatiquement
)
print(f"\n Creation : {resp.status_code}")
print(f"ID cree  : {resp.json()['id']}")

# Headers     authentification par token
headers = {
    "Authorization": "Bearer mon_token_secret",
    "Content-Type":  "application/json",
    "Accept":        "application/json",
}
resp_auth = requests.get(
    "https://api.exemple.com/data",
    headers=headers,
    timeout=10   # toujours mettre un timeout !
)
```

**requests — gestion robuste des erreurs**

```python
import requests
from requests.exceptions import (
    ConnectionError, Timeout,
    HTTPError, RequestException
)

def appeler_api(url, params=None, timeout=10):
    """Appel API robuste avec gestion complete des erreurs."""
    try:
        response = requests.get(
            url, params=params, timeout=timeout
```

```
        )
        response.raise_for_status()    # leve si 4xx/5xx
        return response.json()

    except Timeout:
        print(f"Timeout apres {timeout}s : {url}")
    except ConnectionError:
        print(f"Connexion impossible : {url}")
    except HTTPError as e:
        code = e.response.status_code
        if code == 404:
            print(f"Ressource introuvable (404)")
        elif code == 401:
            print(f"Non autorise (401)     verifier le token")
        elif code == 429:
            print(f"Trop de requetes (429)     ralentir")
        else:
            print(f"Erreur HTTP {code}")
    except RequestException as e:
        print(f"Erreur r seau : {e}")
    return None

# Session     reutiliser la connexion pour performances
with requests.Session() as session:
    session.headers.update({
        "Authorization": "Bearer token_ici",
        "User-Agent":    "MonApp/1.0"
    })
    # Toutes les requetes partagent les headers
    r1 = session.get("https://api.exemple.com/users")
    r2 = session.get("https://api.exemple.com/posts")
```

## Flask — Créer une API REST

**Flask — API REST complète**

```
from flask import Flask, request, jsonify, abort
from datetime import datetime
import uuid

app = Flask(__name__)

# Base de donnees en memoire (pour l'exemple)
etudiants = {
    "1": {"id":"1", "prenom":"Alice", "note":15,
          "ville":"Paris",
          "created": "2024-01-15"},
    "2": {"id":"2", "prenom":"Bob",   "note":18,
```

```
            "ville":"Lyon",
            "created": "2024-01-16"},
}

# GET /api/etudiants      lister tous
@app.route("/api/etudiants", methods=["GET"])
def lister_etudiants():
    ville  = request.args.get("ville")
    note_min = request.args.get("note_min", type=float)

    resultats = list(etudiants.values())

    if ville:
        resultats = [e for e in resultats
                     if e["ville"].lower() == ville.lower()]
    if note_min is not None:
        resultats = [e for e in resultats
                     if e["note"] >= note_min]

    return jsonify({
        "status":  "success",
        "count":   len(resultats),
        "data":    resultats
    })

# GET /api/etudiants/<id>      un seul etudiant
@app.route("/api/etudiants/<string:etudiant_id>",
           methods=["GET"])
def get_etudiant(etudiant_id):
    etudiant = etudiants.get(etudiant_id)
    if not etudiant:
        abort(404, description=f" tudiant  {etudiant_id}
           introuvable")
    return jsonify({"status":"success", "data": etudiant})

# POST /api/etudiants      creer
@app.route("/api/etudiants", methods=["POST"])
def creer_etudiant():
    data = request.get_json()
    if not data:
        abort(400, description="Corps JSON requis")

    champs_requis = ["prenom", "note", "ville"]
    for champ in champs_requis:
        if champ not in data:
            abort(400, description=f"Champ manquant : {champ}")

    nouvel_id = str(uuid.uuid4())[:8]
    etudiant  = {
        "id":      nouvel_id,
```

```python
        "prenom":  data["prenom"],
        "note":    data["note"],
        "ville":   data["ville"],
        "created": datetime.now().strftime("%Y-%m-%d"),
    }
    etudiants[nouvel_id] = etudiant
    return jsonify({"status":"created",
                    "data": etudiant}), 201

# PUT /api/etudiants/<id>     modifier
@app.route("/api/etudiants/<string:etudiant_id>",
           methods=["PUT"])
def modifier_etudiant(etudiant_id):
    if etudiant_id not in etudiants:
        abort(404)
    data = request.get_json()
    etudiants[etudiant_id].update(data)
    return jsonify({"status":"updated",
                    "data": etudiants[etudiant_id]})

# DELETE /api/etudiants/<id>     supprimer
@app.route("/api/etudiants/<string:etudiant_id>",
           methods=["DELETE"])
def supprimer_etudiant(etudiant_id):
    if etudiant_id not in etudiants:
        abort(404)
    del etudiants[etudiant_id]
    return jsonify({"status": "deleted",
                    "id":     etudiant_id})

# Gestionnaire d'erreurs global
@app.errorhandler(404)
def non_trouve(e):
    return jsonify({"status":"error",
                    "message": str(e)}), 404

@app.errorhandler(400)
def requete_invalide(e):
    return jsonify({"status":"error",
                    "message": str(e)}), 400

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

## FastAPI — L'API moderne

**FastAPI — validation automatique et docs**

```python
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, Field, validator
from typing import Optional, List
from datetime import datetime
import uuid

app = FastAPI(
    title="API Etudiants",
    description="Gestion des etudiants ENSEA",
    version="1.0.0"
)

# Modeles Pydantic     validation automatique
class EtudiantBase(BaseModel):
    prenom: str = Field(..., min_length=2, max_length=50)
    note:   float = Field(..., ge=0, le=20)
    ville:  str

    @validator("note")
    def valider_note(cls, v):
        return round(v, 1)

class EtudiantCreer(EtudiantBase):
    pass

class EtudiantReponse(EtudiantBase):
    id:      str
    mention: str
    created: str

    class Config:
        from_attributes = True

# Logique m tier
def calculer_mention(note: float) -> str:
    if note >= 16: return "Tres Bien"
    if note >= 14: return "Bien"
    if note >= 12: return "Assez Bien"
    if note >= 10: return "Passable"
    return "Echec"

# Stockage en memoire
db: dict = {}

# Routes
@app.get("/etudiants",
```

```
          response_model=List[EtudiantReponse],
          summary="Lister tous les   tudiants  ")
async def lister(
    ville:    Optional[str]  = Query(None),
    note_min: Optional[float]= Query(None, ge=0, le=20),
    limit:    int            = Query(50,   ge=1, le=100)
):
    resultats = list(db.values())
    if ville:
        resultats = [e for e in resultats
                     if e["ville"].lower()==ville.lower()]
    if note_min is not None:
        resultats = [e for e in resultats
                     if e["note"] >= note_min]
    return resultats[:limit]

@app.post("/etudiants",
          response_model=EtudiantReponse,
          status_code=201,
          summary="Creer un etudiant")
async def creer(etudiant: EtudiantCreer):
    eid = str(uuid.uuid4())[:8]
    data = {
        **etudiant.dict(),
        "id":      eid,
        "mention": calculer_mention(etudiant.note),
        "created": datetime.now().strftime("%Y-%m-%d"),
    }
    db[eid] = data
    return data

@app.get("/etudiants/{etudiant_id}",
          response_model=EtudiantReponse)
async def obtenir(etudiant_id: str):
    if etudiant_id not in db:
        raise HTTPException(
            status_code=404,
            detail=f"  tudiant  {etudiant_id} introuvable"
        )
    return db[etudiant_id]

@app.delete("/etudiants/{etudiant_id}",
            status_code=204)
async def supprimer(etudiant_id: str):
    if etudiant_id not in db:
        raise HTTPException(status_code=404,
                            detail="Introuvable")
    del db[etudiant_id]

# uvicorn main:app --reload
```

```
# Docs auto : http://localhost:8000/docs
```

**À retenir**

FastAPI génère automatiquement une **documentation interactive** (Swagger UI) accessible à `/docs`. On peut tester tous les endpoints directement dans le navigateur, sans outil externe. La validation Pydantic garantit que les données entrantes sont correctes — et renvoie des erreurs 422 explicites sinon. C'est du niveau production dès le premier jour.

## Web scraping — Extraire des données du web

**BeautifulSoup — parser du HTML**

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

def scraper_citations(url, max_pages=3):
    """Scrape des citations depuis quotes.toscrape.com"""
    toutes_citations = []

    for page in range(1, max_pages + 1):
        print(f"  Page {page}...")
        url_page = f"{url}/page/{page}/"

        try:
            response = requests.get(url_page, timeout=10)
            response.raise_for_status()
        except Exception as e:
            print(f"Erreur page {page}: {e}")
            break

        soup = BeautifulSoup(response.text, "html.parser")

        # Extraire chaque citation
        for quote_div in soup.find_all("div",
                                       class_="quote"):
            texte  = quote_div.find(
                "span", class_="text").get_text()
            auteur = quote_div.find(
                "small", class_="author").get_text()
            tags   = [t.get_text() for t in
                      quote_div.find_all("a", class_="tag")]

            toutes_citations.append({
                "texte":  texte.strip('"'),
```

```
                "auteur": auteur,
                "tags":   ", ".join(tags),
                "page":   page
            })

        time.sleep(1)   # respecter le serveur !

    return pd.DataFrame(toutes_citations)


# Utilisation
df = scraper_citations("https://quotes.toscrape.com")
print(f"Citations collectes : {len(df)}")
print(df[["auteur", "texte"]].head())

# Analyser les auteurs les plus cites
print("\nTop auteurs :")
print(df["auteur"].value_counts().head(5))
```

## Ce qui m'a le plus surpris dans ce chapitre

**Ce qui m'a résisté**

**Trois révélations de ce chapitre :**

**1. FastAPI génère la documentation automatiquement.** Écrire une API avec FastAPI et accéder à `/docs` pour trouver une interface Swagger interactive complète — sans écrire une seule ligne de documentation — m'a stupéfait. Pydantic valide les entrées, FastAPI génère les schémas JSON, et Swagger les affiche. C'est le principe DRY (Don't Repeat Yourself) appliqué à la documentation : le code *est* la documentation.

**2. `raise_for_status()` — une ligne qui sauve des heures.** Sans cette méthode, une requête avec code 404 ou 500 ne lève aucune exception — le programme continue silencieusement avec des données erronées ou vides. `raise_for_status()` transforme les codes d'erreur HTTP en exceptions Python propres. C'est la différence entre un code qui "fonctionne" et un code qui *détecte les problèmes*.

**3. Le web scraping m'a révélé la structure du HTML.** Parser du HTML avec BeautifulSoup m'a appris à voir les pages web comme des structures de données — des arbres de noeuds avec des attributs et des contenus. Ce changement de regard transforme chaque site web en source de données potentielle. Mais avec responsabilité : toujours respecter le `robots.txt` et introduire des délais entre les requêtes.

## Résumé du Chapitre 26

### Ce que j'ai appris dans ce chapitre :

- HTTP : GET, POST, PUT, DELETE — les verbes des APIs REST.
- **requests** : appels HTTP simples, Session, gestion d'erreurs.
- `raise_for_status()` — détecter les erreurs HTTP automatiquement.
- **Flask** : créer une API REST avec routes, JSON, codes HTTP.
- **FastAPI** : validation Pydantic, async/await, docs automatiques.
- Modèles Pydantic : validation des entrées avec `Field` et `validator`.
- **BeautifulSoup** : parser du HTML, extraire des données du web.
- Bonnes pratiques : timeout, délais, `robots.txt`, gestion d'erreurs.

*"Une API, c'est un contrat entre programmes. Bien la concevoir, c'est penser à tous ceux qui l'utiliseront — y compris soi-même dans six mois."*